

Minor additions to C++26 standard library hardening

Document #: P3697R0
Date: 2025-05-15
Project: Programming Language C++
Audience: Library Evolution, Library
Reply-to: Konstantin Varlamov
<varconst@apple.com>
Louis Dionne
<ldionne@apple.com>
Alisdair Meredith
<ameredith1@bloomberg.net>

§ 1 Introduction

This paper is a small follow-up to [P3471R4 “Standard Library Hardening”](#) that hardens the preconditions of a few more library functions. All these functions satisfy the same criteria as the ones in the original paper:

- Violating the precondition results in a memory safety issue (an out-of-bounds access or an access to uninitialized memory);
- The call site has all the necessary data to perform the check;
- The check can be done in constant time and imposes relatively little overhead.

In addition (unlike the “Notable omissions” section in the original paper), these functions are comparatively standalone and don’t involve complex interactions with other members of their respective classes. The additions here are essentially oversights in the original paper that were noticed during offline discussions with various people.

§ 2 List of functions

We propose to harden the relevant preconditions of the functions listed below.

§ 2.1 `basic_stacktrace`

- `current(skip, max_depth)`.

If `skip + max_depth` overflows, an implementation will attempt iterating over a range `[i, j]` where `j < i`, quickly going out of bounds;

— `operator[]`.

This prevents a potential out-of-bounds access and brings the Library closer to an ideal state where an `operator[]` can never cause a memory safety issue in a hardened implementation.

§ 2.2 `shared_ptr<T[N]>`

— `operator[]`.

In the case where the `shared_ptr` points to an array with a statically known size, we are able to perform bounds checking.

§ 2.3 `view_interface`

— `front`;

— `back`.

While we would like to explore hardening Ranges in a dedicated paper, it has been observed that `view_interface::front` and `view_interface::back` are the only front and back functions in the Library that aren't hardened, which might come as an unpleasant surprise to users.

§ 2.4 `counted_iterator`

— `operator*`;

— `operator[]`.

Unlike most iterator types, `counted_iterator` contains *some* of the data necessary to check whether it is valid upon dereference. Specifically, it can check for going out of bounds *past the end* (since it needs to store the remaining length of the sequence and can use 0 as an indicator of reaching the end), but not *before the beginning* (the iterator can be rewound but it does not store the original length and thus cannot check whether it's being rewound past the original beginning). To visualize this, `counted_iterator` can check the right bound but not the left bound. We believe that hardening the preconditions that already exist and can be easily checked is valuable from a pragmatic perspective and preferable to not hardening the class at all.

Note that we are not hardening `operator->`. Unlike with other dereference operators, no immediate access happens within the Library. In the current specification, it is not possible to create an invalid (past the bounds, other than the special “one-past-end”) `counted_iterator` without invoking undefined behavior. Existing preconditions check against the iterator being *incremented* beyond the end; there are no explicit preconditions for *decrementing* the iterator, but an implicit precondition follows from the fact that decrementing the iterator to point before the beginning would result in forming an invalid pointer, which is UB in itself. As mentioned above, for hardening purposes we can only easily check against going past the end (only half of the invariant).

- `counted_iterator(i, n);`
- `operator++;`
- `operator+=;`
- `operator-=.`

These functions don't access the underlying iterator but might end up making length negative, going past the end of the sequence. Do they need to be hardened, given that we also harden the dereference operators? Hardening seems preferable; first, from a more theoretical perspective, maintaining (to the extent possible) the invariant that the iterator is always valid makes it easier to reason about the class; also, setting current to an invalid pointer is undefined behavior in itself. Second, ensuring the iterator is always valid (or one-past-end) also protects the `operator->` for which no explicit precondition currently exists. As stated above, `counted_iterator` only contains data necessary for checking against going past the end but not before the beginning (thus `operator--` is not being hardened).

- `iter_move(counted_iterator);`
- `iter_swap(counted_iterator, counted_iterator).`

While `iter_move` and `iter_swap` don't access memory directly, if the iterator (or one of the iterators) is past the end, it creates a situation where an out-of-bounds access is almost guaranteed and can no longer be easily checked.

§ 2.5 `common_iterator`

- `common_iterator(const common_iterator<I2, S2>&);`
- `operator=;`
- `operator==(const common_iterator&, const common_iterator<I2, S2>&);`
- `operator-(const common_iterator&, const common_iterator<I2, S2>&).`

`common_iterator` is specified in terms of a variant, and some of its functions have a precondition that the variant is not in the invalid state (not `valueless_by_exception`). Thus, unlike variant member functions that are required to handle the case of `valueless_by_exception` gracefully (by throwing or returning a sentinel value), it is undefined behavior to access a `valueless common_iterator` – in practice, an implementation will likely access uninitialized memory. This applies to any modifications to the underlying iterator object, not only dereferences.

- `operator*;`
- `operator->;`
- `operator++;`
- `operator++(int);`

- `iter_move(const common_iterator&);`
- `iter_swap(const common_iterator&, const common_iterator<I2, S2>&).`

Similarly, a few `common_iterator` functions have a precondition that the `common_iterator` holds an iterator and not a sentinel, violating which might also result in a memory safety issue.

§ 3 Proposed wording

§ 3.1 `shared_ptr`

§ 3.1.1 `shared_ptr<T[N]>::operator[]`

Modify 20.3.2.2.6 [[util.smartptr.shared.obs](#)] around paragraph 8 as indicated:

- ```
element_type& operator[](ptrdiff_t i) const;
```
- 8 *Preconditions:* `get() != nullptr` ~~&& i >= 0. If T is U[N], i < N.~~
  - 9 *Hardened preconditions:* `i >= 0`. If T is U[N], `i < N`.
  - 10 *Returns:* `get()[i]`.
  - 11 *Throws:* Nothing.

### § 3.2 `view_interface`

#### § 3.2.1 `view_interface::front`

Modify 25.5.3.2 [[view.interface.members](#)] paragraph 1 as indicated:

- ```
constexpr decltype(auto) front() requires forward_range<D>;
constexpr decltype(auto) front() const requires
    forward_range<const D>;
```
- 1 *Hardened Ppreconditions:* `!empty()` is `true`.
 - 2 *Effects:* Equivalent to: `return *ranges::begin(derived());`

§ 3.2.2 `view_interface::back`

Modify 25.5.3.2 [[view.interface.members](#)] paragraph 3 as indicated:

- ```
constexpr decltype(auto) back() requires bidirectional_range<D>
 && common_range<D>;
constexpr decltype(auto) back() const
 requires bidirectional_range<const D> && common_range<const
 D>;
```
- 3 *Hardened Ppreconditions:* `!empty()` is `true`.

- 4 *Effects*: Equivalent to: `return *ranges::prev(ranges::end(derived()));`

## § 3.3 counted\_iterator

### § 3.3.1 counted\_iterator::operator\*

Modify 24.5.7.4 [counted.iter.elem] paragraph 1 as indicated:

```
constexpr decltype(auto) operator*();
constexpr decltype(auto) operator*() const
 requires dereferenceable<const I>;
```

- 1 *Hardened Ppreconditions*: `length > 0` is `true`.

- 2 *Effects*: Equivalent to: `return *current;`

### § 3.3.2 counted\_iterator::operator[]

Modify 24.5.7.4 [counted.iter.elem] paragraph 4 as indicated:

```
constexpr decltype(auto) operator[](iter_difference_t<I> n)
 const
 requires random_access_iterator<I>;
```

- 4 *Hardened Ppreconditions*: `n < length` is `true`.

- 5 *Effects*: Equivalent to: `return current[n];`

### § 3.3.3 iter\_move(const counted\_iterator&)

Modify 24.5.7.7 [counted.iter.cust] paragraph 1 as indicated:

```
friend constexpr decltype(auto)
 iter_move(const counted_iterator& i)
 noexcept(noexcept(ranges::iter_move(i.current)))
 requires input_iterator<I>;
```

- 1 *Hardened Ppreconditions*: `i.length > 0` is `true`.

- 2 *Effects*: Equivalent to: `return ranges::iter_move(i.current);`

### § 3.3.4 iter\_swap(const counted\_iterator&, const counted\_iterator<I2>&)

Modify 24.5.7.7 [counted.iter.cust] paragraph 3 as indicated:

```
template<indirectly_swappable<I> I2>
 friend constexpr void
 iter_swap(const counted_iterator& x, const
 counted_iterator<I2>& y)
 noexcept(noexcept(ranges::iter_swap(x.current,
 y.current)));
```

3 *Hardened P*preconditions: Both `x.length > 0` and `y.length > 0` are `true`.

4 *Effects*: Equivalent to `ranges::iter_swap(x.current, y.current)`.

### § 3.3.5 `counted_iterator::counted_iterator(I, iter_difference_t<T>)`

Modify 24.5.7.2 [counted.iter.const] paragraph 1 as indicated:

```
constexpr counted_iterator(I i, iter_difference_t<I> n);
```

1 *Hardened P*preconditions: `n >= 0`.

2 *Effects*: Initializes `current` with `std::move(i)` and `length` with `n`.

### § 3.3.6 `counted_iterator::operator++`

Modify 24.5.7.5 [counted.iter.nav] paragraph 1 as indicated:

```
constexpr counted_iterator& operator++();
```

1 *Hardened P*preconditions: `length > 0` is true.

2 *Effects*: Equivalent to:

```
++current;
--length;
return *this;
```

### § 3.3.7 `counted_iterator::operator++(int)`

Modify 24.5.7.5 [counted.iter.nav] paragraph 3 as indicated:

```
constexpr decltype(auto) operator++(int);
```

3 *Hardened P*preconditions: `length > 0` is true.

4 *Effects*: Equivalent to:

```
--length;
try { return current++; }
catch(...) { ++length; throw; }
```

### § 3.3.8 `counted_iterator::operator+=`

Modify 24.5.7.5 [counted.iter.nav] paragraph 10 as indicated:

```
constexpr counted_iterator& operator+=(iter_difference_t<I> n)
requires random_access_iterator<I>;
```

10 *Hardened P*preconditions: `n <= length` is true.

11 *Effects*: Equivalent to:

```
current += n;
length -= n;
return *this;
```

### § 3.3.9 counted\_iterator::operator-=

Modify 24.5.7.5 [counted.iter.nav] paragraph 17 as indicated:

```
constexpr counted_iterator& operator-=(iter_difference_t<I> n)
 requires random_access_iterator<I>;
```

17 *Hardened Ppreconditions*: `-n <= length` *is true*.

18 *Effects*: Equivalent to:

```
current -= n;
length += n;
return *this;
```

## § 3.4 common\_iterator

### § 3.4.1 common\_iterator(const common\_iterator<I2, S2>&)

Modify 24.5.5.3 [common.iter.const] paragraph 3 as indicated:

```
template<class I2, class S2>
 requires convertible_to<const I2&, I> && convertible_to<const
 S2&, S>
 constexpr common_iterator(const common_iterator<I2, S2>& x);
```

3 *Hardened Ppreconditions*: `x.v_.valueless_by_exception()` is *false*.

4 *Effects*: Initializes `v_` as if by `v_{in_place_index<i>, get<i>(x.v_)}`, where `i` is `x.v_.index()`.

### § 3.4.2 common\_iterator::operator=

Modify 24.5.5.3 [common.iter.const] paragraph 5 as indicated:

```
template<class I2, class S2>
 requires convertible_to<const I2&, I> && convertible_to<const
 S2&, S> &&
 assignable_from<I&, const I2&> && assignable_from<S&,
 const S2&>
 constexpr common_iterator& operator=(const
 common_iterator<I2, S2>& x);
```

5 *Hardened Ppreconditions*: `x.v_.valueless_by_exception()` is *false*.

6 *Effects*: Equivalent to:

(6.1) If `v_.index() == x.v_.index()`, then `get<i>(v_) = get<i>(x.v_)`.

(6.2) Otherwise, `v_.emplace<i>(get<i>(x.v_))`.

where `i` is `x.v_.index()`.

### § 3.4.3 `common_iterator::operator*`

Modify 24.5.5.4 [common.iter.access] paragraph 1 as indicated:

```
constexpr decltype(auto) operator*();
constexpr decltype(auto) operator*() const
 requires dereferenceable<const I>;
```

1 *Hardened P* *preconditions*: `holds_alternative<I>(v_)` is `true`.

### § 3.4.4 `common_iterator::operator->`

Modify 24.5.5.4 [common.iter.access] paragraph 4 as indicated:

```
constexpr auto operator->() const
 requires see below;
```

3 The expression in the *requires-clause* is equivalent to:

```
indirectly_readable<const I> &&
(requires(const I& i) { i.operator->(); } ||
 is_reference_v<iter_reference_t<I>> ||
 constructible_from<iter_value_t<I>, iter_reference_t<I>>)
```

4 *Hardened P* *preconditions*: `holds_alternative<I>(v_)` is `true`.

### § 3.4.5 `common_iterator::operator++`

Modify 24.5.5.5 [common.iter.nav] paragraph 1 as indicated:

```
constexpr common_iterator& operator++();
```

1 *Hardened P* *preconditions*: `holds_alternative<I>(v_)` is `true`.

2 *Effects*: Equivalent to `++get<I>(v_)`.

3 *Returns*: `*this`.

### § 3.4.6 `common_iterator::operator++(int)`

Modify 24.5.5.5 [common.iter.nav] paragraph 4 as indicated:

```
constexpr decltype(auto) operator++(int);
```

4 *Hardened P* *preconditions*: `holds_alternative<I>(v_)` is `true`.

5 *Effects*: If `I` models `forward_iterator`, equivalent to:



```
common_iterator tmp = *this;
++*this;
return tmp;
```

### § 3.4.7 `operator==(const common_iterator&, const common_iterator<I2, S2>&)`

Modify 24.5.5.6 [common.iter.cmp] paragraphs 1 and 3 as indicated:

```
template<class I2, sentinel_for<I> S2>
requires sentinel_for<S, I2>
friend constexpr bool operator==(
 const common_iterator& x, const common_iterator<I2, S2>& y);
```

- 1 *Hardened* *P*reconditions: `x.v_.valueless_by_exception()` and `y.v_.valueless_by_exception()` are each `false`.
- 2 Returns: `true` if `i == j`, and otherwise `get<i>(x.v_) == get<j>(y.v_)`, where `i` is `x.v_.index()` and `j` is `y.v_.index()`.

```
template<class I2, sentinel_for<I> S2>
requires sentinel_for<S, I2> && equality_comparable_with<I, I2>
friend constexpr bool operator==(
 const common_iterator& x, const common_iterator<I2, S2>& y);
```

- 3 *Hardened* *P*reconditions: `x.v_.valueless_by_exception()` and `y.v_.valueless_by_exception()` are each `false`.
- 4 Returns: `true` if `i` and `j` are each `1`, and otherwise `get<i>(x.v_) == get<j>(y.v_)`, where `i` is `x.v_.index()` and `j` is `y.v_.index()`.

### § 3.4.8 `operator-(const common_iterator&, const common_iterator<I2, S2>&)`

Modify 24.5.5.6 [common.iter.cmp] paragraph 5 as indicated:

```
template< sized_sentinel_for<I> I2, sized_sentinel_for<I> S2>
requires sized_sentinel_for<S, I2>
friend constexpr iter_difference_t<I2> operator-(
 const common_iterator& x, const common_iterator<I2, S2>& y);
```

- 5 *Hardened* *P*reconditions: `x.v_.valueless_by_exception()` and `y.v_.valueless_by_exception()` are each `false`.
- 6 Returns: `0` if `i` and `j` are each `1`, and otherwise `get<i>(x.v_) - get<j>(y.v_)`, where `i` is `x.v_.index()` and `j` is `y.v_.index()`.

### § 3.4.9 `iter_move(const common_iterator&)`

Modify 24.5.5.7 [common.iter.cust] paragraph 1 as indicated:

```
friend constexpr decltype(auto) iter_move(const common_iterator&
 i)
 noexcept(noexcept(ranges::iter_move(declval<const I&>())))
 requires input_iterator<I>;
```

1 ***Harden******ed*** ***P******re***conditions: holds\_alternative<I>(i.v\_) is true.

2 *Effects*: Equivalent to: return ranges::iter\_move(get<I>(i.v\_));

### § 3.4.10 iter\_swap(const common\_iterator&, const common\_iterator<I2, S2>&)

Modify 24.5.5.7 [common.iter.cust] paragraph 3 as indicated:

```
template<indirectly_swappable<I> I2, class S2>
 friend constexpr void iter_swap(const common_iterator& x,
 const common_iterator<I2, S2>& y)
 noexcept(noexcept(ranges::iter_swap(declval<const I&>(),
 declval<const I2&>())));
```

3 ***Harden******ed*** ***P******re***conditions: holds\_alternative<I>(x.v\_) and holds\_alternative<I2>(y.v\_) are each true.

4 *Effects*: Equivalent to ranges::iter\_swap(get<I>(x.v\_), get<I2>(y.v\_)).

## § 3.5 basic\_stacktrace

### § 3.5.1 basic\_stacktrace::current

Modify 19.6.4.2 [stacktrace.basic.cons] paragraph 5 as indicated:

```
static basic_stacktrace current(size_type skip, size_type
 max_depth,
 const allocator_type& alloc =
 allocator_type()) noexcept;
```

4 Let t be a stacktrace as-if obtained via basic\_stacktrace::current(alloc). Let n be t.size().

5 ***Harden******ed*** ***P******re***conditions: skip <= skip + max\_depth is true.

6 *Returns*: A basic\_stacktrace object where frames\_ is direct-non-list-initialized from arguments t.begin() + min(n, skip), t.begin() + min(n, skip + max\_depth), and alloc, or an empty basic\_stacktrace object if the initialization of frames\_ failed.

### § 3.5.2 basic\_stacktrace::operator[]

Modify 19.6.4.3 [stacktrace.basic.obs] paragraph 10 as indicated:

```
const_reference operator[](size_type frame_no) const;
```

- 10 *Hardened P*reconditions: `frame_no < size()` is `true`.
- 11 Returns: `frames_[frame_no]`.
- 12 Throws: Nothing.

## § 3.6 Feature-test macros

Modify 17.3.2 [\[version.syn\]](#) paragraph 3 as indicated:

- 3 Additionally, each of the following macros are defined in a hardened implementation:

```
#define __cpp_lib_harden_array 202502L // also in
 <array>
+ #define __cpp_lib_harden_basic_stacktrace 202???L // also in
 <stacktrace>
#define __cpp_lib_harden_basic_string 202502L // also in
 <string>
#define __cpp_lib_harden_basic_string_view 202502L // also in
 <string_view>
#define __cpp_lib_harden_bitset 202502L // also in
 <bitset>
+ #define __cpp_lib_harden_common_iterator 202???L // also in
 <iterator>
+ #define __cpp_lib_harden_counted_iterator 202???L // also in
 <iterator>
#define __cpp_lib_harden_deque 202502L // also in
 <deque>
#define __cpp_lib_harden_expected 202502L // also in
 <expected>
#define __cpp_lib_harden_forward_list 202502L // also in
 <forward_list>
#define __cpp_lib_harden_inplace_vector 202502L // also in
 <inplace_vector>
#define __cpp_lib_harden_list 202502L // also in
 <list>
#define __cpp_lib_harden_mdspan 202502L // also in
 <mdspan>
#define __cpp_lib_harden_optional 202502L // also in
 <optional>
+ #define __cpp_lib_harden_shared_ptr_array 202???L // also in
 <memory>
#define __cpp_lib_harden_span 202502L // also in

#define __cpp_lib_harden_valarray 202502L // also in
 <valarray>
#define __cpp_lib_harden_vector 202502L // also in
 <vector>
+ #define __cpp_lib_harden_view_interface 202???L // also in
 <ranges>
```